Information Security

Lecture 3: Symmetric Cryptography

A textbook treatment

Based on the lectures, TPs and quiz of Imane Fouad

October 25, 2026

Contents

How to read this book **2**

4 Symmetric Cryptography **3**
4.1 Cryptosystems, Kerckhoffs, Attack Models 3
4.2 Classical Ciphers and Their Cryptanalysis 4
4.3 XOR and the One-Time Pad 5
4.4 Confusion, Diffusion, Feistel 6
4.5 DES 6
4.6 AES 7
4.7 Modes of Operation 9
4.8 Key Exchange: Diffie–Hellman 10

How to read this book

Each concept in this book is presented in the same rhythm, and every section closes with a *Go deeper* box (the exact details, numbers and edge cases a demanding examiner asks about) followed by *Hard questions, answered.*

1. **Motivation and intuition.** Why do we care? What problem does the idea solve, and what picture should you hold in your head?
2. **Formal statement.** The precise definition or theorem, set in a coloured box so it is easy to find when revising.
3. **Proof.** Every claim is proved. Longer proofs are split into labelled steps, and the one idea that makes the proof work is isolated in a *Key insight* box.
4. **Interpretation and discussion.** What the result really says, where it applies, and which misreadings to avoid.
5. **Examples.** From "trivially small" to "genuinely useful."
6. **Techniques, classic problems and gotchas.** Computational shortcuts, proof patterns, exam favourites with one-line solutions, and the mistakes everyone makes once.

Margin notes carry the informal, "say it out loud" version of what is happening in the main text. Every major section opens with a *What are we trying to get?* box that states the goal in one breath before any mathematics, and closes with a summary box.

Prerequisites. Comfort with binary and hexadecimal, elementary probability, and the modular arithmetic of integers; everything else (finite fields for AES, the number theory behind RSA, Shannon entropy) is built where it is needed.

Chapter 4

Symmetric Cryptography

4.1 Cryptosystems, Kerckhoffs, Attack Models

Definition 4.1.1 (Cryptosystem)**.** *Cryptology* = *cryptography* (secret writing: hiding meaning) + *cryptanalysis* (breaking it). A cryptosystem is (M, C, K, E, D) with $c = E(m, k_e)$ and $m = D(c, k_d)$; *symmetric* if $k_e = k_d$ (everyone who knows k knows the whole secret), *asymmetric* if $k_e \neq k_d$ and k_d is infeasible to derive from k_e. The **key space** is the set of possible keys; **brute force** tries them all. Cryptography provides *confidentiality* (against eavesdroppers), *integrity* (content unaltered) and *authentication* (origin as claimed); ciphers alone give only the first.

Key insight

Kerckhoffs's principle: the system must remain secure when everything except the key is public. Consequences: algorithms are published and reviewed (DES and AES were open competitions); a compromised key is replaced without redesign; "secret algorithm" is a red flag because algorithms leak and cannot be rotated. Modern practice: public, sophisticated algorithm, short secret key (the inverse of the historical "simple algorithm, long key").

Definition 4.1.2 (Attack models)**.** What the cryptanalyst has besides the algorithm and the target ciphertext: **ciphertext only** (COA); **known plaintext** (KPA: some plaintext–ciphertext pairs under the key); **chosen plaintext** (CPA: plaintexts of the attacker's choice encrypted); **chosen ciphertext** (CCA: ciphertexts of the attacker's choice decrypted); **chosen text** (both). Power increases COA < KPA < CPA, CCA; security under a stronger model implies the weaker. Ways to attack: *analytical* (cleverness: weaknesses in the algorithm or its use), *brute force* (money: parallel search; *computationally secure* = cost to crack exceeds the value), *social engineering*, *implementation attacks* (software, hardware, side channels).

Hard questions, answered

Q. Give a realistic setting for each attack model.
A. COA: intercepted radio. KPA: encrypted files with known headers (PDF magic bytes). CPA: a web app that encrypts a cookie value the attacker controls. CCA: a server that decrypts submitted tokens and reveals whether

padding was valid.

Q. Classify: power-trace analysis of a smart card; bribing an operator; trying all 2^{56} DES keys; exploiting ECB block independence.
A. Implementation; social engineering; brute force; analytical.

4.2 Classical Ciphers and Their Cryptanalysis

Definition 4.2.1 (Caesar, substitution, Vigenère)**. Caesar**: $c = (p + k) \bmod 26$; key space 26. **Simple substitution**: a fixed permutation of the alphabet; key space $26! \approx 4.03 \times 10^{26}$. **Vigenère**: a keyword $k_1 \dots k_L$ repeats; letter i is Caesar-shifted by $k_{i \bmod L}$.

Theorem 4.2.2 (Why substitution falls to frequency analysis)**.** *A monoalphabetic substitution permutes the letter-frequency histogram of the plaintext but does not change its shape; the ciphertext letter with frequency $\approx 12.7\%$ is the image of E, and so on. Hence the key is recovered letter by letter from statistics, in time independent of the key-space size.*

Proof. Each plaintext letter a maps to one fixed ciphertext letter $\sigma(a)$, so $\#\{i : c_i = \sigma(a)\} = \#\{i : p_i = a\}$: the multiset of counts is preserved and only relabelled. English counts are far from uniform (E 12.7, T 9.1, A 8.2, O 7.5, I 7.0, N 6.7, S 6.3, H 6.1, R 6.0…; J, X, Q, Z rarest), so the relabelling is largely determined by sorting; short words, doubled letters and repeated patterns resolve ties. □

Technique: Breaking Vigenère with known or guessed key length L

Split the ciphertext into L groups `text[i::L]`, $i = 0..L-1$; every letter in group i was shifted by the same k_i, so each group is a Caesar cipher (Theorem 4.2.2 applies within the group). In each group take the most frequent letter, assume it is E, shift = (that letter − E) mod 26, key letter = the shift (E→G: shift 2, key C). If L is unknown: repeated trigrams occur at distances that are multiples of L (Kasiski), or compare the index of coincidence for candidate L. The attack needs the key to *repeat*; a random key as long as the message and never reused leaves one letter per group and no statistics (the one-time pad).

Worked example: TP, warm-up and Room 1

Caesar $K = 4$: `CRYPTO IS FUN UNTIL THE PROFESSOR SAYS QUIZ` → `GVCTXS MW JYR YRXMP XLI TVSJIWWSV WECW UYMD`. $K = 8$ decrypt: `GWCL JMBBMZ VWB JM CAQVO BWWTA BW AWTDM BPQA` → `YOUD BETTER NOT BE USING TOOLS TO SOLVE THIS`. Unknown key, brute force 26 shifts: `ESP VPJ TD FYVYZHY ECJ MCFEP QZCNP` is shift 11: `THE KEY IS UNKNOWN TRY BRUTE FORCE`. Substitution table (A→Q, B→M, C→J, D→Z, E→T, …): `KEEP THE SECRET SAFE` → `LTTN EKT YTJRTE YQGT`; `ERVYE DXVR SXFPJ` → `TRUST YOUR LOGIC`. Slide: `GSRH RH Z HVXIVG` is the reversed alphabet (Atbash): `THIS IS A SECRET`.

Go deeper: the details an examiner asks about

- Key-space size is necessary, not sufficient: 26! keys and minutes to break. The exam question "is a large key space enough?" wants Theorem 4.2.2.
- Enigma: German naval advantage early in WWII; broken by Turing's *bombe*; the lesson the slides draw is that with computers substitution and transposition alone are obsolete, and modern ciphers *combine* substitution with repeated transposition so that statistical relations vanish (DES, AES).
- "Computationally secure" is economic (cost > value); "unconditionally secure" (one-time pad) is information-theoretic.

Hard questions, answered

Q. Why is substitution with 26! keys broken faster than Caesar with 26 keys by brute force?
A. Analytical attack vs exhaustive search: the statistics survive the permutation; a few dozen informed guesses beat 26 trial decryptions only in the sense that both are trivial, but the point is that 26! gave no security at all.

Q. Two messages XOR-encrypted with the same one-time key. What does Eve get?
A. $c_1 \oplus c_2 = m_1 \oplus m_2$; with structured text she recovers both (crib dragging). Reuse destroys the pad.

4.3 XOR and the One-Time Pad

XOR: "different → 1, same → 0"; $(x \oplus k) \oplus k = x$, so the same operation encrypts and decrypts. `Hello` ⊕ repeating key `key` gives `00100011 00000000 00010101 00000111 00001010`. With a key that is random, as long as the message, and never reused, this is the *one-time pad*: each ciphertext is equally likely under every plaintext of that length, so COA learns nothing. With a *repeating* key it is Vigenère on bytes and breaks under every model: KPA gives plaintext ⊕ ciphertext = key; CPA with all-zero plaintext returns the key; CCA with all-zero ciphertext returns the key; COA sees that the second ciphertext byte is `00000000`, hence the second key byte equals the second plaintext byte, and continues from language structure.

Hard questions, answered

Q. Why must modern ciphers resist *chosen*-plaintext attacks rather than just known-plaintext?
A. Attackers routinely control inputs (they can make a server encrypt their data); a scheme that leaks under CPA is broken in practice even if KPA is safe, and CPA security implies the weaker models.

4.4 Confusion, Diffusion, Feistel

Definition 4.4.1 (Confusion and diffusion (Shannon))**. Confusion**: make the relation between key, plaintext and ciphertext complex (nonlinear); Caesar's $c = (p + k) \bmod 26$ has none. **Diffusion**: spread the influence of each input bit over many output bits so that local statistics vanish. Block ciphers achieve them by rounds of substitution (confusion) and permutation/mixing (diffusion).

Theorem 4.4.2 (Feistel networks invert for free)**.** *Let a round map $(L_{i-1}, R_{i-1}) \mapsto (L_i, R_i)$ with $L_i = R_{i-1}$, $R_i = L_{i-1} \oplus f(R_{i-1}, K_i)$ for* any *function f. Then (L_{i-1}, R_{i-1}) is recovered by $R_{i-1} = L_i$, $L_{i-1} = R_i \oplus f(L_i, K_i)$. Hence an n-round Feistel cipher is decrypted by the same algorithm with the round keys in reverse order, and f need not be invertible.*

Proof. L_i is a copy of R_{i-1}; substituting into R_i and XOR-ing $f(R_{i-1}, K_i)$ once more cancels it since $x \oplus y \oplus y = x$. Apply round by round from $i = n$ down to 1. □

4.5 DES

Definition 4.5.1 (DES)**.** Block 64 bits, key 56 bits (the 64-bit input key has 8 parity bits at positions 8, 16, …, 64, dropped by *permuted choice* PC-1), 16 Feistel rounds with distinct 48-bit subkeys, an initial permutation IP before and its inverse after. The round function $f(R, K_i)$, 32 bits in, 32 out: (1) **expansion** E: 32 → 48 bits by duplicating the two boundary bits of each 4-bit group; (2) XOR with K_i; (3) eight **S-boxes** $S_1..S_8$, each 6 bits → 4: *row* = outer bits b_1b_6, *column* = inner bits $b_2b_3b_4b_5$; the only nonlinear step; (4) **permutation** P: a fixed rearrangement that sends each S-box's output bits to different S-boxes next round (diffusion). IP, E and P are position tables: they move bits, never change values.

Worked example: One round, slide and toy

Slide: input `0x0123456789ABCDEF`; IP's first row (58, 50, 42, 34, 26, 18, 10, 2) selects bits giving first output byte `11001100`; R_0 = `0xF0AAF0AA`; $E(R_0)$ = `011110 100001 010101 010101 011110 100001 010101 010101`; after $\oplus K_1$: `0x6117BA866527`; first group `011000`: row 00 = 0, column 1100 = 12, $S_1[0, 12] = 5 =$ `0101`; all boxes: `0x5C82B597`; P: `0x234AA9BB`. Toy ($L_0 =$ `1010`, $R_0 =$ `1001`, $E' = [4, 1, 2, 3, 4, 1]$, $K_1 =$ `101011`, $P' = [2, 4, 3, 1]$): $E'(R_0) =$ `110011`, $\oplus K_1 =$ `011000`, $S_1 \to$ `0101`, $P' \to$ `1100`, $L_1 = R_0 =$ `1001`, $R_1 = L_0 \oplus$ `1100` = `0110`.

Definition 4.5.2 (Key schedule and decryption)**.** PC-1 keeps 56 bits, split into 28-bit halves C_0, D_0. Round i: rotate each half left by 1 bit in rounds 1, 2, 9, 16 and by 2 bits otherwise; PC-2 selects 48 of the 56 as K_i (each K_i is a permuted subset of the key). Total rotation $4 \cdot 1 + 12 \cdot 2 = 28$, so $C_{16} = C_0$, $D_{16} = D_0$. Decryption (Theorem 4.4.2): the same 16 keys in order $K_{16} \dots K_1$; equivalently run the schedule with no rotation in round 1, right rotations of 1 in rounds 2, 9, 16 and of 2 elsewhere. No new keys are computed. Toy:

key `10101101`, PC-1 $= [1,2,3,5,6,7] \to$ `101110`, $C_0 =$ `101`, $D_0 =$ `110`; round 1 shift 1: `011`, `101`; PC-2 $= [1,2,4,5]$ on `011101` $\to K_1 =$ `0110`; round 2 continues from C_1, D_1: `110`, `011` $\to$ `110011` $\to K_2 =$ `1101`.

Interpretation & discussion

Strength: $2^{56} \approx 7.2 \times 10^{16}$ keys; at 10^9 keys/s, 7.2×10^7 s ≈ 2.3 years on one machine; the EFF's \$250,000 machine (1998) cracked DES in under 2 days. DES died of key size, not of cryptanalysis. **3DES** applies DES three times (EDE, so $K_1 = K_2 = K_3$ degrades to DES for compatibility) with 2 keys (112 bits effective) or 3 (168); double DES is pointless because *meet in the middle* (encrypt a known plaintext under all K_1, decrypt its ciphertext under all K_2, match) costs $\approx 2^{57}$, not 2^{112}. 3DES survives in legacy finance (ATMs); AES replaced it (and is 5× faster in CPU hardware).

Go deeper: the details an examiner asks about

- Why DES decrypts with reversed keys and AES does not: Theorem 4.4.2 needs the Feistel structure; AES's operations act on the whole block and need explicit inverses.
- S-box indexing: "outside bits = row, inside bits = column"; row 2 of S_1 is 4 1 14 8 13 6 2 11 15 12 9 7 3 10 5 0.
- Cipher table: DES 1975 (56/64), 3DES 1995 (112/168, 64), Twofish, Serpent, Rijndael 1998 (128/192/256, 128); AES standardised 2001 (FIPS-197).

Hard questions, answered

Q. S_1 on input `100110`?
A. Row $b_1b_6 =$ `10` $= 2$, column $b_2..b_5 =$ `0011` $= 3$: $S_1[2,3] = 8 =$ `1000`.

Q. Why is the effective key 56 bits when 64 are entered, and why does 2DES give ≈ 57 bits?
A. Eight parity bits are discarded by PC-1. Meet in the middle turns the 2^{112} search into two 2^{56} tables matched in the middle.

4.6 AES

Definition 4.6.1 (GF(2^8) arithmetic)**.** Bytes are polynomials of degree < 8 over $\{0,1\}$; addition is XOR; multiplication is polynomial multiplication reduced modulo $m(x) = x^8 + x^4 + x^3 + x + 1$ (`0x11B`). Multiplying by x (`02`, `xtime`): shift left one bit, and if the bit shifted out was 1, XOR with `1B` (that is $x^8 \equiv x^4 + x^3 + x + 1$). $03 \cdot b = (02 \cdot b) \oplus b$. Every nonzero byte has an inverse, which is what makes the S-box and MixColumns invertible.

Definition 4.6.2 (AES-128)**.** Rijndael (Rijmen, Daemen; FIPS-197, 2001); block 128 bits as a 4×4 byte *state* filled column-wise (b_{i+4j} at row i, column j); keys 128/192/256 with 10/12/14 rounds and $N_r + 1$ round keys; a substitution–permutation network, *not* Feistel. Encryption: AddRoundKey(K_0); rounds

1..$N_r - 1$ of **SubBytes** (each byte through the S-box; confusion), **ShiftRows** (row r rotated left by r; moves bytes *between* columns), **MixColumns** (each column multiplied by $\begin{smallmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{smallmatrix}$ in GF(2^8); mixes *within* a column), **AddRoundKey** (XOR with K_i); final round without MixColumns. ShiftRows and MixColumns together give diffusion: after two rounds every byte depends on every input byte. Decryption: AddRoundKey(K_{10}); for rounds 9..1: InvShiftRows (right rotations), InvSubBytes (inverse S-box), AddRoundKey, InvMixColumns (matrix with `0E 0B 0D 09`); then InvShiftRows, InvSubBytes, AddRoundKey(K_0). Key expansion: 44 words; $w_0..w_3$ = key; for $i \equiv 0 \pmod 4$: RotWord, SubWord, XOR `Rcon`; always XOR with w_{i-4}; Rcon first bytes `01 02 04 08 10 20 40 80 1B 36` (powers of 02 in the field, hence `1B` after `80`).

Worked example: MixColumns, first byte of column `D4 BF 5D 30`

$02 \cdot$ D4: `11010100` $\rightarrow$ `10101000` = `A8`, carry out, $\oplus$`1B` = `B3`. $02 \cdot$ BF: `10111111` $\rightarrow$ `01111110` = `7E`, carry, $\oplus$`1B` = `65`; $03 \cdot$ BF = $65 \oplus$ BF = DA. b_0 = B3 $\oplus$ DA $\oplus$ 5D $\oplus$ 30 = 69 $\oplus$ 5D $\oplus$ 30 = 34 $\oplus$ 30 = 04. Whole column $\rightarrow$ `04 66 81 E5`; AddRoundKey: `04`$\oplus$`A0` = `A4`. SubBytes examples: `19`$\rightarrow$`D4`, `00`$\rightarrow$`63`, `03`$\rightarrow$`7B`, `71`$\rightarrow$`A3`. ShiftRows of the index state: row 1 `11 12 13 10`, row 2 `22 23 20 21`, row 3 `33 30 31 32`. AddRoundKey with all-`0F` key: XOR each byte with `0F` (`23`$\rightarrow$`2C`).

From the TP: AES without diffusion

Two rounds on one byte, $c = S(S(p \oplus k_0) \oplus k_1) \oplus k_2$, with ShiftRows and MixColumns removed. From $(00, \text{A4})$: $k_2 = \text{A4} \oplus S(S(k_0) \oplus k_1)$, a function of (k_0, k_1); enumerate the 2^{16} pairs, derive k_2, keep those consistent with $(01, \text{F8})$ and $(02, 3\text{F})$. Each byte position is attacked *independently* with three known plaintexts; with diffusion the state must be attacked as a whole (2^{128}). Confusion without diffusion is attackable byte by byte.

Go deeper: the details an examiner asks about

- The final round omits MixColumns because a trailing public linear map adds no security and would spoil the symmetry between encryption and decryption.
- SubBytes is the only nonlinear step; without it AES is a linear map solvable by linear algebra.
- AES-128 vs 256: key length, rounds (10 vs 14), round keys (11 vs 15), expansion details; block size and the four operations unchanged.

Hard questions, answered

Q. Compute $02 \cdot 57$ and $03 \cdot 57$.
A. `57`=`01010111` $\rightarrow$ `10101110` = `AE`, no carry: `AE`; $03 \cdot 57$ = AE $\oplus$ 57 = F9.

Q. Which step gives confusion, which diffusion, what breaks without each?
A. SubBytes: confusion (remove: linear, trivial). ShiftRows+MixColumns: diffusion (remove: byte-wise 2^{16} attack). AddRoundKey: the key (remove:

no secret).

4.7 Modes of Operation

Definition 4.7.1 (ECB, CBC, CTR)**.** A block cipher handles one block; a *mode* extends it. **ECB**: $C_i = E_K(P_i)$; parallel both ways; *deterministic per block*, so equal plaintext blocks give equal ciphertext blocks and the *structure* leaks (the penguin). **CBC**: $C_i = E_K(P_i \oplus C_{i-1})$, $C_0 = IV$; $P_i = D_K(C_i) \oplus C_{i-1}$; encryption sequential, decryption parallel with random access; the **IV** is random, *not secret* (sent in clear) and prevents a repeated first block from repeating its ciphertext. **CTR**: $C_i = P_i \oplus E_K(\text{nonce}\|i)$; a stream cipher from a block cipher; fully parallel; the nonce must never repeat under a key (else a two-time pad). Others: CFB, OFB, CCM (CTR for confidentiality + CBC-MAC for integrity: authenticated encryption).

Theorem 4.7.2 (Error propagation in CBC)**.** *A single flipped bit in ciphertext block C_i garbles plaintext block P_i entirely and flips exactly that bit in P_{i+1}; later blocks are unaffected.*

Proof. $P_i = D_K(C_i) \oplus C_{i-1}$ passes the corrupted block through D_K (pseudorandom output). $P_{i+1} = D_K(C_{i+1}) \oplus C_i$ XORs the corrupted C_i directly. P_{i+2} uses C_{i+1}, C_{i+2} only. □

From the TP: OpenSSL mission 2, ECB on an image

OpenSSL's salted format: 8-byte signature `Salted__` + 8-byte *salt* (public; the key is derived from password and salt by PBKDF2), so skip 16 bytes. The image is $432 \times 475 = 205{,}200$ bytes $= 12{,}825$ blocks exactly, so PKCS#7 adds one full padding block: ciphertext 205,216, file 205,232 bytes. Counting equal 16-byte blocks (`collections.Counter`) and colouring equal blocks alike reproduces the picture's outline: a *pattern, not the secret*. `openssl enc -d -aes-256-ecb -pbkdf2 -p -in classified.enc -out recovered.raw`; re-encrypting with `-aes-256-cbc` leaves essentially no repeated blocks because each block is XOR-ed with the previous ciphertext first.

Hard questions, answered

Q. Why is the IV sent in clear not a weakness, and what would be?
A. The IV only needs to be unique/unpredictable so equal first blocks differ; knowing C_0 does not help invert E_K. Reusing an IV, or a predictable one under CPA, is the weakness.

Q. Rank ECB, CBC, CTR: encryption parallel / decryption parallel / random access / hides patterns.
A. ECB yes/yes/yes/no; CBC no/yes/yes/yes; CTR yes/yes/yes/yes (unique nonce).

Q. Strong cipher, weak system: AES-256-ECB on 16-byte customer IDs.
A. Equal IDs give equal ciphertexts; frequency and equality analysis works without touching AES. Use CBC/CTR with fresh IVs or authenticated encryption.

4.8 Key Exchange: Diffie–Hellman

Symmetric ciphers need the key delivered over a *secure channel*. Merkle (1974, idea rejected by his professor; 1975 paper rejected by CACM as "extremely dangerous to transmit key information in the clear") and Diffie–Hellman (1976) showed it is not needed. Public g, p; Alice picks a, sends $A = g^a \bmod p$; Bob picks b, sends $B = g^b \bmod p$; both compute $g^{ab} = B^a = A^b \bmod p$. Eve sees g, p, A, B and must solve the **discrete logarithm** (slide: $6^x \bmod 13 = 2$, i.e. $x = 5$; $6^x \bmod 13 = 9$, $x = 4$), trivial for $p = 13$, infeasible for 2048-bit p. DH gives a shared secret, not authentication: a *man in the middle* negotiates k_1 with Alice and k_2 with Bob and relays everything; endpoints must be authenticated (Chapter 5).

Hard questions, answered

Q. Which hard problem underlies DH, and which underlies RSA?
A. Discrete logarithm; integer factorisation.

Q. Does encrypting the DH messages with Bob's public key stop the MITM?
A. Only if Alice holds Bob's *authentic* public key; if Eve can substitute hers, she decrypts and relays. The problem is key authenticity: certificates.

Summary: Symmetric cryptography

- Kerckhoffs: only the key is secret. Models COA < KPA < CPA/CCA.
- Substitution dies to frequency analysis (statistics survive relabelling); Vigenère splits into L Caesar groups; a non-repeating random key is the one-time pad.
- Confusion (nonlinear S-boxes) + diffusion (permutation/mixing); Feistel inverts for free with reversed keys.
- DES: 64/56, 16 rounds, E–XOR–S-box–P, rotations 1,2,9,16; brute-forced in 1998; 2DES falls to meet-in-the-middle.
- AES: GF(2^8) with `1B`; SubBytes, ShiftRows, MixColumns, AddRoundKey; 10/12/14 rounds; SPN needs explicit inverses.
- ECB leaks structure; CBC chains with a public random IV; CTR is a parallel stream; CCM adds integrity.
- DH: discrete log; no authentication, hence MITM.